

贪心算法

《算法设计与分析》

信息科学与技术学院 曾华琳



04



数据压缩 (Huffman 编码)



四、数据压缩 (Huffman 编码)

- **数据压缩问题** – 串 S 和 S' :
 - $S \rightarrow S' \rightarrow S$, 则 $|S'| < |S|$
- Huffman 编码是一种有效而且被广泛应用的数据压缩技术; 一般可压缩掉 20% to 90%, 这取决于被压缩数据的特征。
- 假定数据为一系列字符, Huffman 贪心算法使用了一张字符频度表, 根据它来构造一种将每个字符表示成二进串的最优方式。



四、数据压缩 (Huffman 编码)

- 字符编码问题。

假设我们有一个包含100,000(a-f)个字符的数据文件要压缩存储。各字符的出现频度见下表：

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
Frequency	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

- 我们可以编码为多少位呢？

固定长度编码： $100,000 \times 3 = 300,000$

可变长度编码： $(45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4) \times 1,000 = 224,000 \text{ bits}$



四、数据压缩 (Huffman 编码)

- **前缀编码：**没有一个编码是另一个编码的前缀
 - 可以证明，由字符编码获得的最优数据压缩总可用某种前缀编码来获得。
- **编码：**将文件中每个字符的编码并置起来即可
 $abc \rightarrow 0 \cdot 101 \cdot 100 \rightarrow 0101100.$
- **前缀编码的好处在于它简化了编码与解码**
 $001011101 \rightarrow 0 \cdot 0 \cdot 101 \cdot 1101 \rightarrow aabe.$

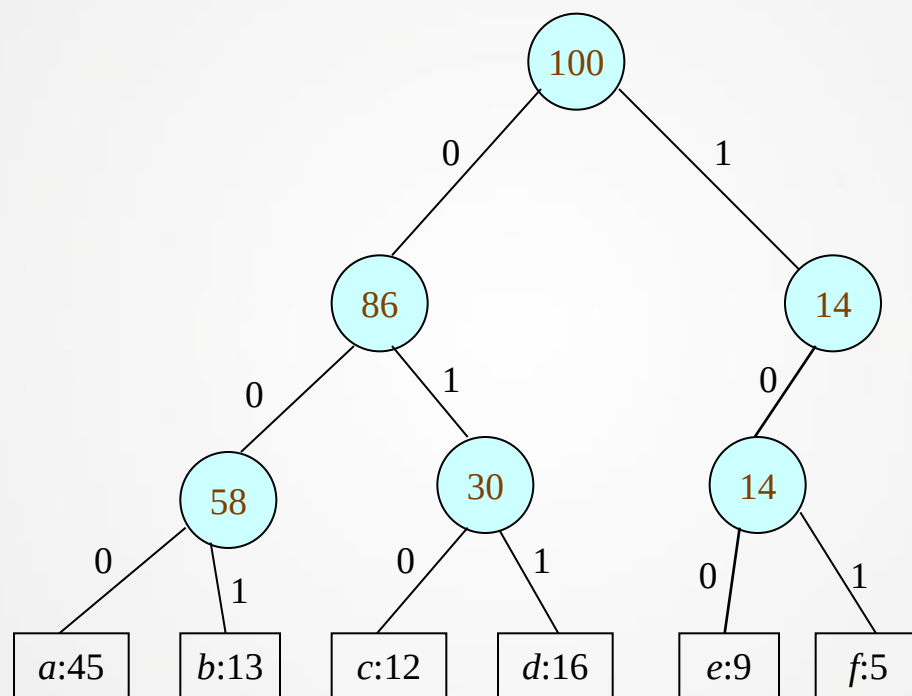


四、数据压缩 (Huffman 编码)

- 解码过程需要有一种关于前缀编码的方便的表示。使得初试编码可以很容易地被识别出来。
- 有一种表示法就是叶子为给定字符的二叉树
- 我们将一个字符的编码解释为从根至该字符的路径，其中0表示“转向左子节点”，1表示“转向右子节点”。则我们可以构造出一棵与编码序列相符的二叉树。

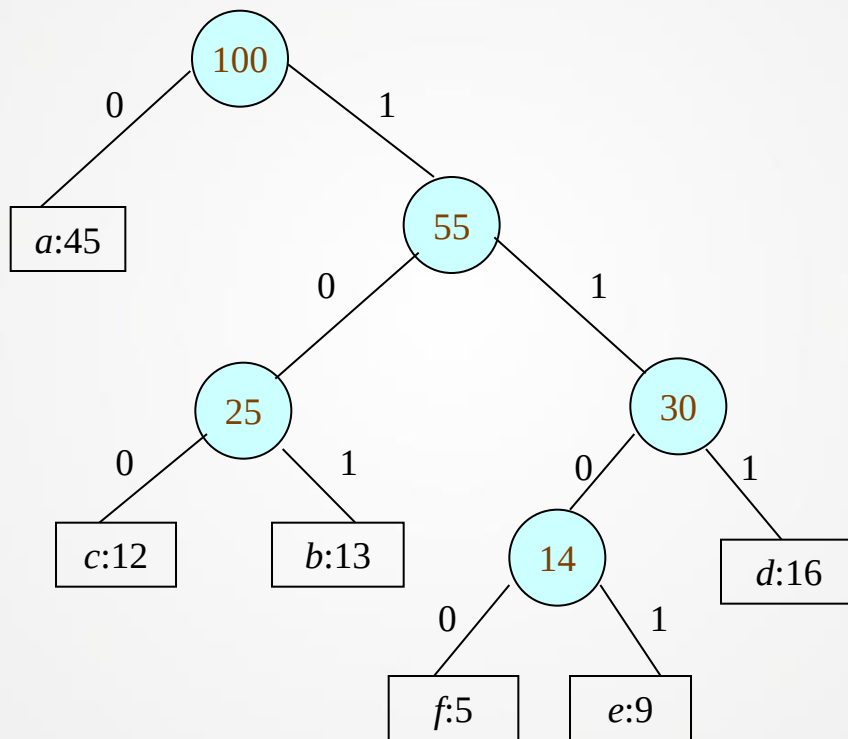


最优前缀编码





最优前缀编码





树 T 的代价

- 给定对应一种前缀编码的二叉树 T ，很容易计算出编码一个文件所需的位数。对字母表 C 中的每个字符 c ，设 $f(c)$ 表示 c 在文件中出现的频度， $d_T(c)$ 表示 c 的叶子在树中的深度。注意 $d_T(c)$ 也是字符 c 的编码的长度。这样，编码一个文件所需的位数就是

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

我们把它定义为**树 T 的代价**。

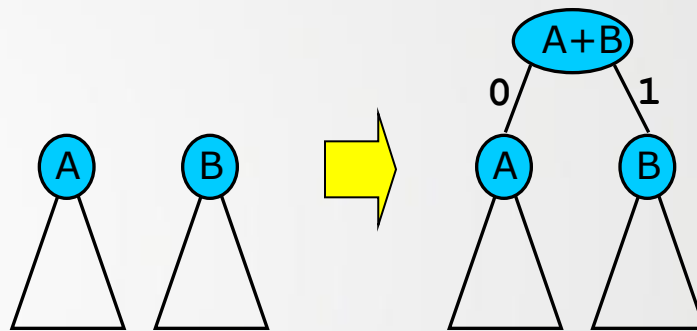


Huffman 算法 – 概念

- Huffman 算法，自底向上构造编码。

考虑森林：

- 最初 – 每个节点都是独立的。
- 每一步 – 把两棵树合并为一棵



- 重复至只剩一棵树。
- 怎么合并？贪心选择 – 选择频度最低的树！



构造 Huffman 编码

HuffmanCode(C)

```
1   $Q \leftarrow C$ 
2  for  $i \leftarrow 1$  to  $n - 1$  do
3      allocate a new node  $z$ 
4       $x \leftarrow \text{ExtractMin}(Q)$ 
5       $y \leftarrow \text{ExtractMin}(Q)$ 
6       $f(z) \leftarrow f(x) + f(y)$ 
7      Insert( $Q, z$ )
8  return ExtractMin( $Q$ ) //返回树的根.
```



构造 Huffman 编码

f:5

e:9

c:12

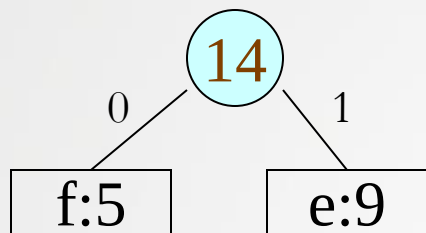
b:13

d:16

a:45



构造 Huffman 编码



c:12

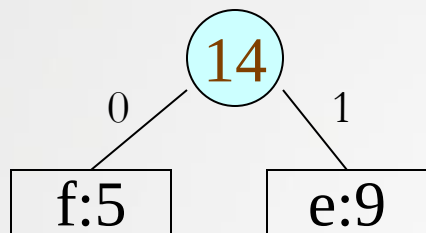
b:13

d:16

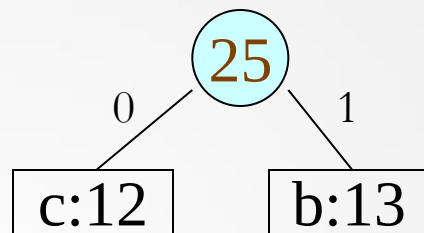
a:45



构造 Huffman 编码



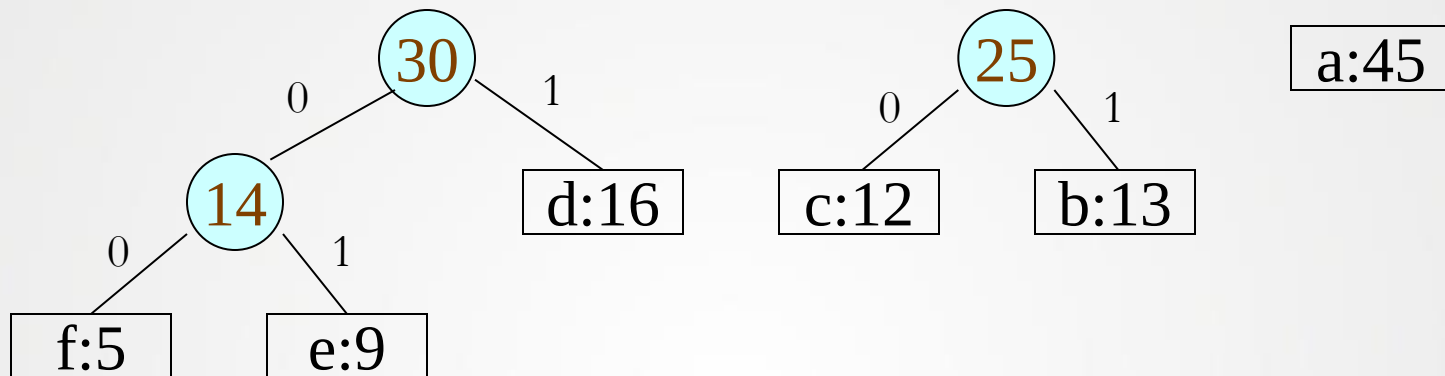
d:16



a:45



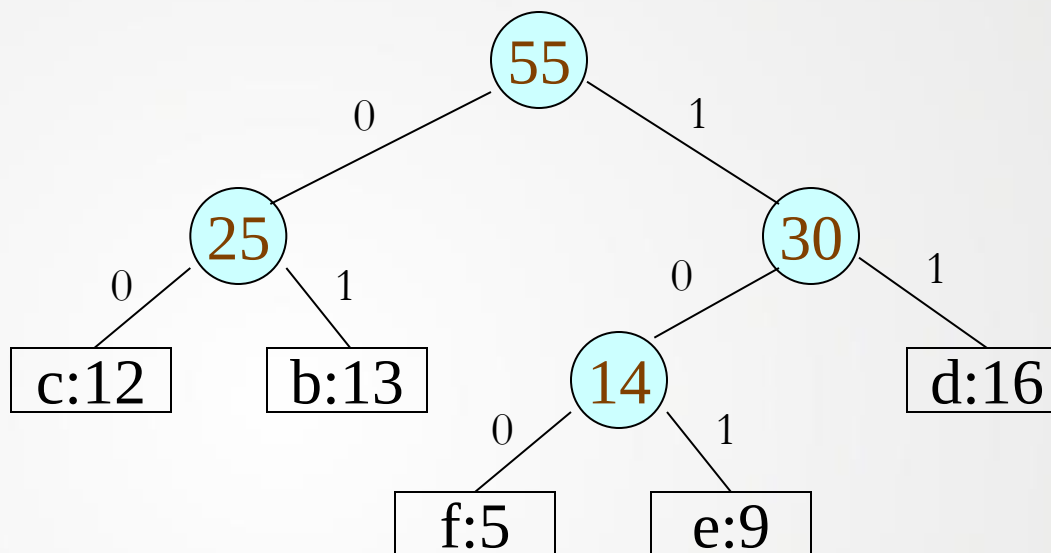
构造 Huffman 编码





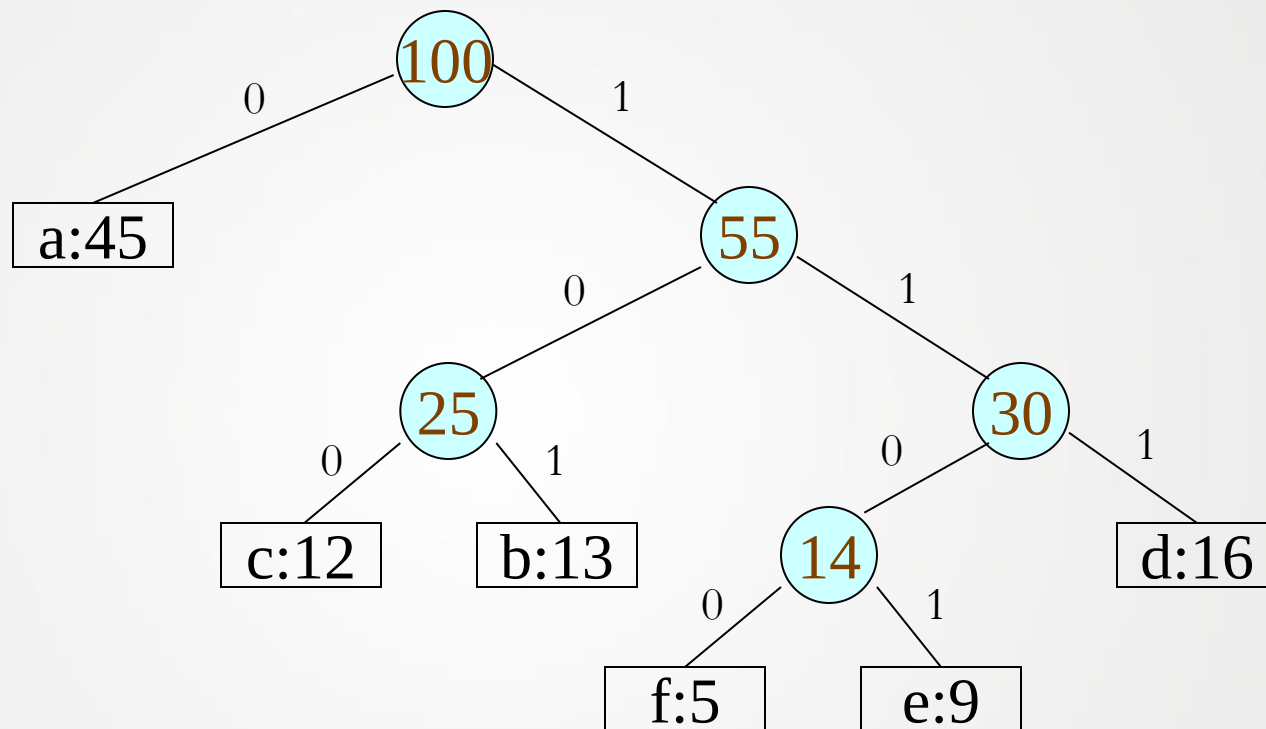
构造 Huffman 编码

a:45





构造 Huffman 编码



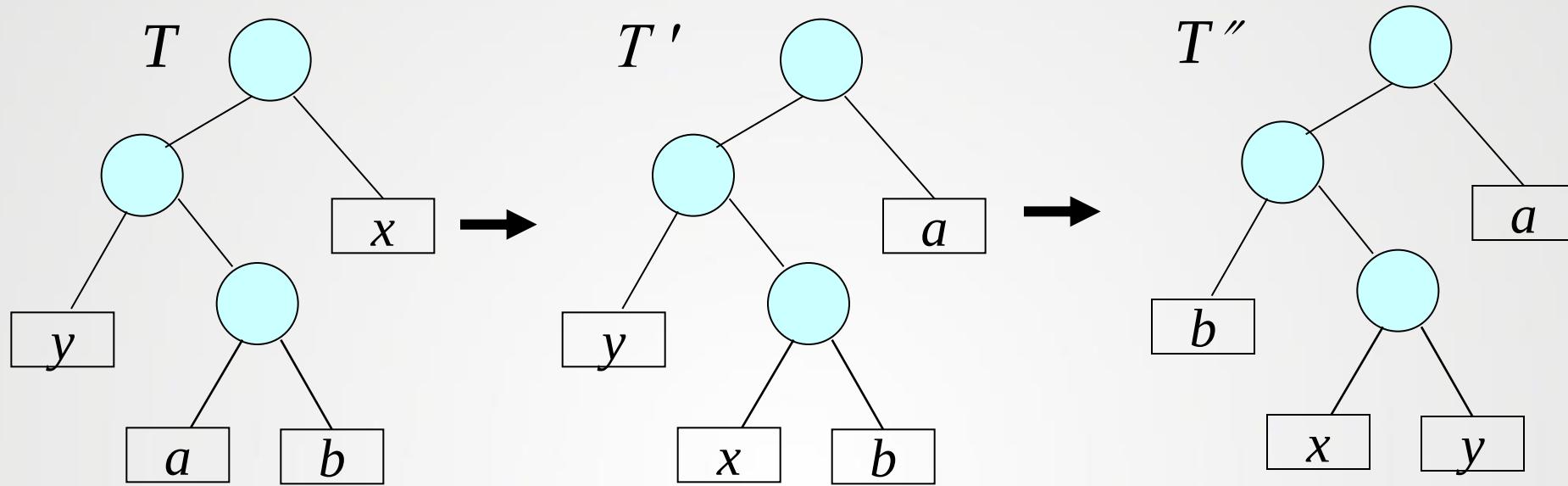


Huffman算法的正确性

■ 定理7.4 :

令 C 为一个字符表。对任意的 $c \in C$ ，字符 c 在文件中出现的频率为 $f(c)$ 。令 x 和 y 为 C 中出现频率最小的两个字符。那么对 C 存在一个最优前缀编码。在这种编码中， x 和 y 的编码长度最长，且长度相等，只有最后一位不同。

■ 证明:



$$\begin{aligned}
 B(T) - B(T') &= \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \\
 &= f(x) d_T(x) + f(a) d_T(a) - f(x) d_{T'}(x) - f(a) d_{T'}(a) \\
 &= f(x) d_T(x) + f(a) d_T(a) - f(x) d_T(a) - f(a) d_T(x) \\
 &= (f(a) - f(x))(d_T(a) - d_T(x))
 \end{aligned}$$



Huffman算法的正确性

- 故 $B(T') \leq B(T)$, 又有 $B(T'') \leq B(T')$, 则 $B(T'') \leq B(T)$, 因为 T 为最优, 故 $B(T) \leq B(T'')$, 结论: $B(T'') = B(T)$.
- 此引理证明了构造最优前缀编码有贪心选择特征。下面的引理将证明构造最优前缀代码的问题具有最优子结构性质。



Huffman算法的正确性

定理7.5

- 令 C' 为一个字符表。对任意的 $c \in C$ ，字符 c 在文件中出现的频率为 $f(c)$ 。令 x 和 y 为 C 中出现频率最小的两个字符。从中删除 x 和 y 两个字符并加入新的字符 z ，得到新的字符表 $C' = \{C - \{x, y\}\} \cup \{z\}$ 。除了 $f(z) = f(x) + f(y)$ 外， C' 中字符频率的定义同 C 。树 T' 代表 C' 的最优前缀编码，对树的叶子节点 z ，添加两个孩子节点 x 和 y ，就可以得到树 T ，则 T 代表 C 的最优前缀编码。



Huffman算法的正确性

证明:

- 设 T 为 C 的一种最优前缀编码。

$$f(x)d_T(x) + f(y)d_T(y) = (f(x) + f(y))(d'(z) + 1) = f(z)d'(z) + (f(x) + f(y)),$$

$$\text{故 } B(T) = B(T') + f(x) + f(y)$$

若 T' 表示 C' 上的一种非最优前缀编码, 则存在树 T'' 使得 $B(T'') < B(T')$ 。 设 z 为 x, y 的父节点 且有 $f(z) = f(x) + f(y)$, 则我们可以构造树 T''' 。 然后



Huffman算法的正确性

$B(T''') = B(T'') + f[x] + f[y]$ 然后 因为 $B(T'') < B(T')$,

所以 $B(T''') = B(T'') + f[x] + f[y] < B(T') + f[x] + f[y]$

因此 $B(T) = B(T') + f[x] + f[y]$,

即 $B(T''') < B(T)$,

$B(T''') = B(T'') + f(x) + f(y)$.

又因为 $B(T'') < B(T')$,

所以 $B(T''') = B(T'') + f(x) + f(y) < B(T') + f(x) + f(y)$

因而 $B(T) = B(T') + f(x) + f(y)$,

故 $B(T''') < B(T)$,

这与 T 的最优性矛盾。所以 T' 表示 C 一种最优前缀编码。

- 由引理7.4和引理7.5直接可得HuffmanCode(C)产生一种最优前缀编码。

谢谢大家

